

OBJECT-ORIENTED PROGRAMMING

Final Term Project

OOP Analysis of

Python's pathlib Library

with Custom SmartPath Extension

Group Members

Name	Student ID
[Muhammad Shehrooz Nawaz]	[F25BDATS1M02066]
[Muhammad Ibadullah Bhutta]	[F25BDATS1M02070]
[Shahzaib Asif]	[F25BDATS1M02073]

1. Library Overview

1.1 What is pathlib?

Python's `pathlib` module, introduced in Python 3.4 via PEP 428, provides an object-oriented API for working with filesystem paths. Instead of treating paths as plain strings — as the older `os` and `os.path` modules do — `pathlib` wraps every path inside a class. This gives paths meaningful methods and properties (like `.name`, `.stem`, `.suffix`, `.parent`) and allows operations using the familiar `/` operator.

`pathlib` ships as part of the Python Standard Library, which means no installation is required. Any Python 3.4+ environment has it available immediately with a simple import.

```
from pathlib import Path
```

1.2 Real-World Use Cases

- Data science pipelines — building cross-platform file paths to datasets, CSVs, and model checkpoints without hardcoded separators
- DevOps and automation scripts — walking directory trees, finding files by pattern (`glob`/`rglob`), moving and renaming files
- Web frameworks (Django, FastAPI) — resolving static files, media directories, and template paths portably
- Testing (pytest) — the popular `tmp_path` fixture returns a `pathlib.Path` object
- Config management — constructing platform-appropriate config and log file paths

1.3 Installation

No installation needed. `pathlib` is bundled with Python 3.4 and later:

```
# Verify Python version
python3 --version    # needs 3.4+

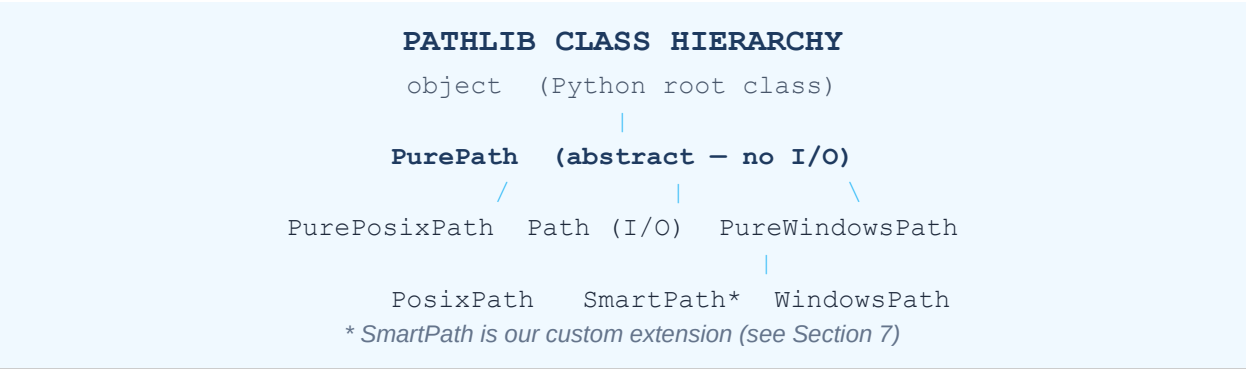
# Import in your script
from pathlib import Path, PurePath, PurePosixPath, PureWindowsPath
```

1.4 Who Uses It?

`pathlib` is used by virtually every major Python project: NumPy, pandas, scikit-learn, Django, Flask, pytest, and the Python standard library itself all reference `pathlib.Path` internally. It replaced `os.path` as the recommended approach in modern Python codebases after Python 3.6.

2. Class Hierarchy Diagram

The `pathlib` library uses a carefully designed multi-level class hierarchy. The diagram below shows the full inheritance chain, from Python's root object down to our custom `SmartPath` extension.



2.1 Key Classes Explained

Class	Layer	Purpose
object	Python root	Universal base of all Python objects
PurePath	Abstract base	Pure path manipulation — no filesystem I/O. Defines parts, drive, root, parent, name, stem, suffix
PurePosixPath	Concrete pure	POSIX semantics: forward slash, no drive letter. Works on Linux/Mac
PureWindowsPath	Concrete pure	Windows semantics: backslash, drive letters (C:\). Works cross-platform
Path	Concrete I/O	Inherits PurePath + adds stat(), read_text(), write_text(), glob(), mkdir(), iterdir() etc.
PosixPath	Platform leaf	Auto-selected on Linux/Mac when you call Path()
WindowsPath	Platform leaf	Auto-selected on Windows when you call Path()
SmartPath (*ours*)	Custom extension	Subclass of Path adding checksum(), word_count(), metadata(), safe_copy(), backup(), find_duplicates()

3. OOP Principles Analysis

3.1 Inheritance

Inheritance is the most visible OOP principle in pathlib. The library uses a two-level hierarchy: a pure (computation-only) layer and a concrete (I/O-enabled) layer. This design choice means PurePath subclasses can be used safely in any environment — even on a Windows machine where you need to reason about POSIX paths without actually touching the filesystem.

Code Example — Inheritance chain in action:

```
from pathlib import Path, PurePath

p = Path('/home/student/project/report.pdf')
```

```
# Path's full MRO (Method Resolution Order):
print(Path.__mro__)
# => (<class 'Path'>, <class 'PurePath'>, <class 'object'>)

# Path inherits ALL PurePath methods:
print(p.name)      # 'report.pdf'    -- from PurePath
print(p.stem)      # 'report'       -- from PurePath
print(p.suffix)    # '.pdf'         -- from PurePath
print(p.parent)    # /home/student/project

# ... AND adds its own I/O methods:
print(p.exists())  # True / False
print(p.stat().st_size) # file size in bytes
```

The platform-specific dispatch is a clever use of `__new__`: when you call `Path('/some/path')`, Python automatically returns a `PosixPath` on Linux/macOS or a `WindowsPath` on Windows — without you needing to know which one. This is polymorphic object creation through inheritance.

3.2 Encapsulation

Encapsulation means hiding internal implementation details and exposing only a clean, safe public interface. `pathlib` is an excellent example of this principle: before `pathlib`, Python programmers had to call `os.getcwd()`, `os.path.join()`, `os.stat()`, `open()`, and `os.path.splitext()` as separate, unrelated functions. `pathlib` wraps all of these inside a single `Path` object.

Code Example — Encapsulation of low-level os calls:

```
from pathlib import Path

f = Path('/tmp/demo.txt')
f.write_text('Hello, pathlib!') # wraps open() + write + close

# Encapsulated:           What it hides internally:
f.read_text()             # open(f, 'r').read()
f.stat().st_size           # os.stat(f).st_size
f.exists()                # os.path.exists(f)
f.is_file()               # os.path.isfile(f)
list(f.parent.glob('*.txt')) # os.scandir + fnmatch
```

Private internals in `pathlib` include `__str_normcase` (normalised string for case-insensitive comparison on Windows), `__flavour` (stores platform-specific parsing rules), and `__accessor` (the low-level system call adapter). These are marked private by convention (leading underscore) and are never part of the public API.

3.3 Polymorphism

Polymorphism allows different classes to respond to the same interface. In `pathlib`, polymorphism appears in three forms:

- Subtype polymorphism — `PurePosixPath` and `PureWindowsPath` share the same method names (`.parts`, `.drive`, `.name`, etc.) but return different results based on their platform semantics
- Operator overloading — the `/` operator is overloaded (`__truediv__`) on all `PurePath` subclasses so `p / 'subdir' / 'file.txt'` works uniformly
- Duck typing — any function that accepts a `PurePath` argument can work with `Path`, `PosixPath`, `WindowsPath`, or `SmartPath`

Code Example — Polymorphic behaviour:

```
from pathlib import PurePosixPath, PureWindowsPath, PurePath

def describe_path(p: PurePath) -> None:
    # This function works with ANY PurePath subclass
    print(f'Type: {type(p).__name__}')
    print(f'Drive: {p.drive!r}')
    print(f'Parts: {p.parts}')

posix = PurePosixPath('/usr/local/bin/python')
win = PureWindowsPath(r'C:\Users\student\file.py')

describe_path(posix)    # Drive: ''    Parts: ('/', 'usr', ...)
describe_path(win)     # Drive: 'C:'  Parts: ('C:\\', 'Users', ...)

# Operator polymorphism - / works on all subclasses:
child = Path('/home') / 'user' / 'docs'    # PosixPath('/home/user/docs')
```

3.4 Abstraction

Abstraction means modelling only what is relevant and hiding unnecessary complexity. `pathlib` abstracts away the fact that paths are strings with platform-specific separator characters. A user of `pathlib` never needs to think about `os.sep`, forward vs. backslash, or manual string concatenation.

Code Example — Abstraction over directory traversal:

```
from pathlib import Path

project = Path('/home/student/project')

# HIGH-LEVEL ABSTRACTION: iterdir() hides os.scandir()
for item in project.iterdir():
    print(item.name, '->', 'dir' if item.is_dir() else 'file')

# ABSTRACTION: glob() hides os.walk() + fnmatch
for py_file in project.rglob('*.py'):    # recursive glob
    print(py_file.relative_to(project))

# ABSTRACTION: open() wraps the built-in open() context manager
with (project / 'output.txt').open('w') as fh:
```

```
fh.write('Saved via pathlib abstraction')
```

4. Design Decision Analysis

4.1 The PurePath / Path Split

The most significant design decision in pathlib is the separation between PurePath (pure computation, no I/O) and Path (I/O operations). This is an application of the Single Responsibility Principle: one class handles path string manipulation; another adds filesystem access.

4.2 Why This Decision Was Made

- Cross-platform path reasoning: on a Linux machine you sometimes need to manipulate Windows paths (e.g., when parsing a config file created on Windows). PureWindowsPath lets you do this without touching the filesystem.
- Testability: unit tests can construct and manipulate PurePath objects without creating real files, making tests faster and side-effect free.
- Safety: Pure path operations cannot accidentally read or write files, which prevents bugs in code that only needs to compute paths.

4.3 Trade-offs

Advantages	Disadvantages
Clean separation of concerns: pure manipulation vs. I/O	Two class hierarchies can confuse new users (which one do I use?)
PurePath subclasses work on any OS for any target platform	isinstance checks require knowing the full hierarchy
Enables dependency injection and mocking in tests	Cannot add I/O features to PurePath without changing the hierarchy

4.4 Alternative Design: Protocol-Based (Structural Subtyping)

An alternative would be to define a PathLike protocol (similar to os.PathLike) and let any object that implements `__fspath__()` participate in path operations, without requiring class inheritance. This is the approach used by many modern languages (e.g., Rust's `AsRef<Path>`). The advantage is greater flexibility; the disadvantage is losing the organised method discovery that the class hierarchy provides.

A second alternative would be a single flat Path class with an `io=False` parameter to disable I/O methods. This is simpler but loses the type-safety benefit: code accepting PurePath explicitly signals 'I will not touch the filesystem', which a flat class cannot express.

Overall, the PurePath/Path split is a well-justified design decision that prioritises correctness and cross-platform safety over simplicity.

5. Comparison with Alternative: os.path

5.1 Overview of os.path

Before pathlib, Python developers used `os` and `os.path` — a collection of functions that treat paths as plain strings. It is still part of the standard library and works well for simple scripts, but it lacks the object-oriented design of `pathlib`.

Feature	pathlib (OOP)	os.path (Functional)
Paradigm	Object-Oriented: Path objects with methods	Procedural: standalone functions
Path joining	<code>p / 'subdir' / 'file.txt'</code>	<code>os.path.join(p, 'subdir', 'file.txt')</code>
Read a file	<code>Path('f.txt').read_text()</code>	<code>open('f.txt').read()</code>
Get stem	<code>Path('a.b.txt').stem</code> → 'a.b'	<code>os.path.splitext('a.b.txt')[0]</code>
Cross-platform	<code>PureWindowsPath</code> works on Linux	<code>os.path</code> is always current OS
Readability	High — method chaining, property access	Low — nested function calls
Extensibility	Subclass <code>Path</code> to add features	Cannot extend — just functions
Available since	Python 3.4	Python 1.x

5.2 OOP Design Comparison

The fundamental difference is that `os.path` treats paths as data (strings), while `pathlib` treats them as objects with behaviour. This is the core OOP insight: combine data and the operations that act on it into a single unit. `pathlib`'s approach leads to more readable, self-documenting, and extensible code. The ability to subclass `Path` (as we do with `SmartPath`) is impossible with the functional `os.path` approach.

6. Custom Extension: SmartPath

6.1 Design Goals

`SmartPath` subclasses `pathlib.Path` to add data-science-friendly utilities. The design goals were:

- Stay 100% backward compatible — every existing `Path` method works unchanged
- Demonstrate all four OOP principles inside our own class
- Add genuinely useful features: file hashing, word counting, metadata export, safe copying, backups, and duplicate detection
- Ensure the `/` operator returns `SmartPath` (not plain `Path`) so the entire chain is 'smart'

6.2 Class Definition and `__new__` Override

Subclassing Path requires overriding `__new__` (not `__init__`) because Path uses `__new__` internally for its platform-specific dispatch mechanism (returning `PosixPath` vs `WindowsPath`). Without this, Python 3.9 and earlier raise a `TypeError`.

```
class SmartPath(Path):
    def __new__(cls, *args, **kwargs):
        obj = super().__new__(cls, *args, **kwargs)
        return obj
```

6.3 Key Methods

`checksum(algorithm='md5')`

Returns the hex digest of a file. Reads the file in 64 KB chunks to be memory-efficient for large files — important in data science where datasets can be gigabytes in size.

```
def checksum(self, algorithm: str = 'md5') -> str:
    self._require_file()
    h = hashlib.new(algorithm)
    with self.open('rb') as fh:
        for chunk in iter(lambda: fh.read(65536), b''):
            h.update(chunk)
    return h.hexdigest()
```

`metadata()` → dict

Returns a comprehensive JSON-ready dictionary combining `stat()`, `checksum()`, and `word_count()`. This method demonstrates composition — it calls multiple methods to build a richer result.

```
smartp = SmartPath('report.txt')
print(smartp.metadata())
# {'name': 'report.txt', 'size_bytes': 1024,
#  'modified_at': '2026-05-11T09:30:00',
#  'checksum_md5': 'abc123...', 'word_stats': {...}}
```

`safe_copy(destination, overwrite=False)`

Copies a file to a destination, automatically appending `_1`, `_2`, etc. to the filename if a file already exists there. Returns a `SmartPath` pointing to the actual destination file.

`find_duplicates(algorithm='md5')`

Scans a directory tree and groups files with identical checksums. Returns a dict mapping hash → list of duplicate `SmartPaths`. This is the kind of utility that takes 30+ lines with `os.walk` but is clean and readable as a `SmartPath` method.

`__truediv__` Override

The `/` operator is overridden to ensure that `SmartPath / 'subdir'` returns a `SmartPath` (not a plain `Path`). Without this override, navigating into subdirectories would 'downgrade' the object.

```
def __truediv__(self, key) -> 'SmartPath':
    result = super().__truediv__(key)
```

```
return SmartPath(result) # re-wrap as SmartPath
```

6.4 OOP Principles in SmartPath

Principle	How SmartPath demonstrates it
Inheritance	SmartPath IS-A Path IS-A PurePath — inherits all ~40 methods without rewriting them
Encapsulation	<code>_require_file()</code> is a private guard method; checksum internals (chunk reading, hashlib) are hidden
Polymorphism	<code>__truediv__</code> override; <code>describe_path()</code> works on SmartPath just as on plain Path
Abstraction	<code>metadata()</code> composes <code>stat/checksum/word_count</code> into one call; users never call hashlib directly

7. References

- Python Software Foundation. pathlib — Object-oriented filesystem paths. Python 3.12 Documentation. <https://docs.python.org/3/library/pathlib.html>
- Python Software Foundation. PEP 428 — The pathlib module — object-oriented filesystem paths. <https://peps.python.org/pep-0428/>
- CPython source code — pathlib module. https://github.com/python/cpython/blob/main/Lib/pathlib/__init__.py
- Reitz, K. & Schlusser, T. The Hitchhiker's Guide to Python: Best Practices for Development. O'Reilly Media. (pathlib chapter)
- Real Python. Python's pathlib Module: Taming the File System. <https://realpython.com/python-pathlib/>
- Gamma, E., Helm, R., Johnson, R., Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
- Python Software Foundation. os.path — Common pathname manipulations. <https://docs.python.org/3/library/os.path.html>